DESIGN SPECIFICATIONS DOCUMENT

System Architecture & Technical Design

Project: VOO Ward Citizen Engagement Platform

Version: 2.0.0

Date: April 2, 2026

DESIGN SPECIFICATIONS DOCUMENT

VOO Ward Citizen Engagement Platform

Project Name: VOO Ward (Kyamatu Ward) Citizen Engagement Platform

Version: 2.0.0

Date: April 2, 2026

Prepared By: VOO Ward Development Team

Document Type: Design Specifications

---

1. Introduction

1.1 Purpose

This document provides the detailed design specifications for the VOO Ward Citizen Engagement Platform. It describes the system architecture, module designs, database schemas, API contracts, security mechanisms, and deployment strategies utilized to construct and maintain the platform.

1.2 Scope

The VOO Ward platform is a multi-channel citizen service system serving the residents of Kyamatu Ward. It enables civic participation through four access channels:

- **USSD**: Accessible to feature phones via short code `*384#`

- **Web Admin Dashboard**: For ward administrators and the Member of County Assembly (MCA)

- **Citizen Portal API**: Self-service REST APIs for citizens

- **WhatsApp Integration**: Conversational access via Twilio webhook

- **Mobile Application**: React Native/Expo application for smartphones

1.3 Intended Audience

- Software Developers and System Maintainers

- Academic Reviewers and Examiners

- Quality Assurance Engineers

- System Administrators and Operations Personnel

---

2. System Overview

2.1 Product Perspective

The VOO Ward platform is a backend-centric system engineered using Node.js and Express.js. It exposes HTTP endpoints consumed by multiple channels and client applications. The system supports both feature phones and smartphones to ensure maximum accessibility.

2.2 Key Design Principles

1. **Multi-Channel Accessibility**: A unified backend infrastructure serves USSD, web, mobile, and WhatsApp clients.

2. **Graceful Degradation**: The system is designed to return fail-safe messages rather than terminating processes during component failures.

3. **Multilingual Support**: Supports English, Swahili, and Kamba languages.

4. **Role-Based Access Control**: Strict segregation of duties between Super Administrator and standard Administrator roles.

5. **Modular Architecture**: Clear separation of routing, service, middleware, and database layers.

6. **Security by Design**: Implementation of rate limiting, input sanitization, and cryptographic hashing.

---

3. Architecture Design

3.1 Architectural Pattern

The system follows a layered monolithic architecture with a clear separation of concerns:

1. **Presentation Layer**: USSD Navigation, Admin Dashboard, Citizen Portal, WhatsApp Interface

2. **Routing Layer**: Express Route Handlers

3. **Middleware Layer**: Security, Rate Limiting, Authentication, Error Handling

4. **Service Layer**: AI Services, Localization, Cache Management, Session Management

5. **Data Access Layer**: Database queries and Object-Relational Mappings

6. **Database Layer**: PostgreSQL, MongoDB, and Redis instances

3.2 Component Interaction

The Express backend acts as the central orchestrator. External actors (Feature Phones, Web Browsers, Mobile Apps, WhatsApp clients) interact with specific input vectors. The application processes these requests through standard middleware, executes business logic within the service layer, and persists state in the database layer.

4. Module Design

4.1 USSD Module

Purpose: Provides text-based menu-driven access for feature phone constituents.

Component: `ussdCore.js` and `routes/ussd.js`

Functionality: Manages session state via the provider's `sessionId`, returning `CON` (continue) or `END` (terminate) strings. Includes fallback language localization functions.

4.2 Admin Dashboard Module

Purpose: Comprehensive administration panel.

Component: `admin-dashboard.js`

Functionality: Facilitates authentication via hashed PINs. Enables CRUD operations for announcements, issues, users, and bursary applications. Ensures session management and data export functions.

4.3 Citizen Portal Module

Purpose: Self-service API suite for authenticated citizens.

Component: `routes/citizenPortal.js`

Functionality: OTP-based authentication yielding a short-lived Bearer token. Exposes authenticated endpoints for issue reporting and bursary tracking.

4.4 WhatsApp Integration Module

Purpose: Conversational logic handler.

Component: `routes/whatsapp.js`

Functionality: Processes Twilio webhooks, manages in-memory conversational state, and returns TwiML formatted XML responses.

4.5 Mobile Application Module

Purpose: Native smartphone client.

Component: React Native / Expo Application

Functionality: Facilitates voter registration, issue reporting with geocoding, and application tracking through an authenticated REST API session.

5. Database Design

5.1 Database Strategy

The platform implements polyglot persistence to optimize data access patterns:

- **PostgreSQL**: Stores structured relational data (Administrators, Bursary Applications, Geographic Areas).

- **MongoDB**: Stores unstructured or highly variable document data (Constituents, Issues, Announcements).

- **Redis**: Serves as an ephemeral store for session caching and rate limits.

5.2 PostgreSQL Schema Structure

- `admin_users`: Manages internal credentials and Role-Based Access Control parameters.

- `bursary_applications`: Tracks financial requests with strict status enumerations.

- `areas`: Standardizes geographic tracking hierarchies.

5.3 MongoDB Collections Structure

- `constituents`: Tracks registered users, their verification status, and demographic parameters.

- `issues`: Stores citizen reports, geospatial coordinates, media references, and resolution timelines.

- `announcements`: Maintains public notices with priority markers.

---

6. API Design

6.1 API Standards

All APIs follow RESTful conventions, returning JSON payloads with standardized HTTP status codes. Authentication is maintained via JSON Web Tokens passed in the Authorization header.

6.2 Primary Endpoints

- **Health Checks**: `/health`, `/health/detailed`

- **USSD Aggregator Webhook**: `POST /ussd`

- **Administrator Interfaces**: `POST /admin/api/login`, `GET /admin/api/issues`

- **Citizen Interfaces**: `POST /api/citizen/request-otp`, `GET /api/citizen/bursaries`

- **WhatsApp Webhook**: `POST /api/whatsapp/webhook`

---

7. Security Design

7.1 Authentication Mechanisms

The system employs varying authentication paradigms based on the access channel:

- **Administrative Channel**: Static Username and PIN combined with bcrypt hashing (10 rounds).

- **Citizen Channel**: Dynamic OTP generation, verified to issue a 24-hour JSON Web Token.

- **USSD/WhatsApp Channels**: Relies on telecom or gateway verification of the MSISDN.

7.2 Application Security

Standardized middlewares apply the following constraints:

- Global request rate limiting to mitigate denial-of-service attempts.

- Cryptographic session token generation.

- Strict cross-origin resource sharing (CORS) enforcement.

- Data sanitization to prevent injection vulnerabilities.

---

8. Deployment Architecture

8.1 Environmental Configuration

The service leverages a 12-factor application methodology, deferring configuration to environment variables (`PORT`, `DATABASE_URL`, `JWT_SECRET`).

8.2 Production Environment

The system is deployed within a cloud environment utilizing Docker containerization. It interfaces with external managed databases (Supabase for PostgreSQL, Atlas for MongoDB, Redis Labs for caching). Operational metrics and crash analytics are centrally logged.

---

9. Technology Stack

- **Runtime**: Node.js 18+

- **Framework**: Express.js

- **Databases**: PostgreSQL 15+, MongoDB 6.x, Redis 4.x

- **Cryptographic Library**: bcryptjs

- **Third-Party Integrations**: Twilio (WhatsApp)

- **Mobile Development Environment**: React Native, Expo

End of Design Specifications Document
